

Initial Experiments: Extending the MVP Simulator

William Dennis

June 2026

Extends the MVP simulator (`docs/mvp/mvp_specification.tex`) by enriching the state space, action space, and reward function, moving toward the full MDP defined in `docs/design/initial_design.tex`.

1 State Space Extension

References `docs/design/initial_design.tex` §3 and Issue #124.

1.1 Enriched StateView

1.2 Contextual Bandit Conditioning

1.3 Open Questions

2 Action Space Extension

References `docs/design/initial_design.tex` §3 and Issue #126.

2.1 ActionConfig

2.2 Transition Model Updates

2.3 Agent Compatibility

2.4 Open Questions

3 Reward Function Extension

References `docs/design/initial_design.tex` §3 and Issue #132. *Implementation: PR 1c (feat/1c-reward) — merged.*

3.1 RewardWeightsConfig

An optional `reward_weights` block controls the delayed reward component. When absent the reward is purely per-state (MVP backward-compatible mode):

$$r(s, a, t) = m[t \bmod S] \cdot r(s')$$

When present, the schema accepts these fields:

Field	Type	Default
<code>mode</code>	<code>Literal["multi_timescale"]</code>	<code>"multi_timescale"</code>
<code>delayed_reward_interval</code>	<code>int ≥ 1</code>	21
<code>delayed_reward_value</code>	<code>float ≥ 0</code>	10.0
<code>delayed_reward_scale</code>	<code>float None</code>	None
<code>delayed_reward_threshold</code>	<code>float ∈ [0,1] None</code>	None

All numeric fields are validated at load time via Pydantic `Field` constraints. The mode is constrained to `Literal["multi_timescale"]`; when additional modes are added later the union can be expanded.

3.2 Flat Bonus (Option A)

When `delayed_reward_scale` is `None` a flat bonus fires at every interval boundary:

$$r_{\text{bonus}} = \begin{cases} \text{value} & \text{if } t > 0 \text{ and } t \equiv 0 \pmod{N} \\ 0 & \text{otherwise} \end{cases}$$

where $N = \text{delayed_reward_interval}$ and t is the zero-indexed cumulative step count (`state.global_step`).

3.3 Scaled Bonus (Option B)

When `delayed_reward_scale` is set the bonus depends on the fraction of steps in the interval where the simulated user was in the active state:

$$\text{rate} = \frac{\text{active_count}}{N} \quad r_{\text{bonus}} = \begin{cases} \text{scale} \times \text{rate} & \text{if } \text{rate} \geq \text{threshold} \\ 0 & \text{otherwise} \end{cases}$$

The active count is tracked in `CompoundReward._active_count`, incremented on every `reward()` call where `state.activity == "active"` and reset to zero at each interval boundary.

3.4 Episode Boundaries

The `RewardHandler ABC` now defines a `reset()` method that clears episode-local state (`_active_count`). `Environment.reset()` calls `self._reward.reset()` so that reusing a reward instance across episodes does not leak state.

3.5 Results

The 1c experiment config is at `docs/initial_experiments/configs/1c_reward_only.yaml`. Mean total rewards across 50 seeds (seeds 0–49) for each agent and reward mode. The runner is at `docs/initial_experiments/run_1c_experiments.py`.

Agent	Mean total reward (50 seeds)				
	No bonus	Flat bonus	Scaled bonus	Flat – base	Scaled – base
Thompson Sampling	160.1	369.5	163.0	209.4	2.9
Contextual TS	160.1	369.5	163.0	209.4	2.9
ϵ -greedy	157.5	356.6	160.1	199.1	2.6
Contextual EG	157.5	356.6	160.1	199.1	2.6
UCB	162.9	356.3	166.1	193.4	3.2
Contextual UCB	162.9	356.3	166.1	193.4	3.2
Decaying ϵ	161.1	354.2	164.1	193.1	3.0
Contextual DEC	161.1	354.2	164.1	193.1	3.0
Random	136.3	346.3	138.0	210.0	1.7

All contextual variants use `contextual=True` with `context_feature='activity'`. Contextual and non-contextual agents produce identical totals because the hand-tuned transition matrix does not depend on context values in a way the contextual bandits can exploit differently.

The flat bonus (interval=21, value=10.0) adds roughly 193–210 in total reward across all agents, consistent with the expected $\lfloor 450/21 \rfloor = 21$ firings $\times 10.0 = 210$ bonus over a 450-step episode. The scaled bonus (scale=5.0, threshold=0.6) adds only 2–3 on average: no agent sustains a 60% active rate within a 21-step interval given the hand-tuned transition probabilities. Thompson Sampling achieves the highest base reward (160.1) and benefits most from the flat bonus (369.5 total).

3.6 Open Questions

- The `delayed_reward_interval` and `delayed_reward_value` parameters are placeholders. Calibration against real-world user data would inform clinically meaningful values.
- The scaled bonus threshold (default 0.6) and scale (used as 5.0 in this run; schema default is `None`) were chosen arbitrarily. A grid search could identify regions where the delayed reward shapes agent behaviour without overwhelming the per-step signal.
- The flat vs. scaled choice is a config toggle today. A future extension could blend both: a fixed bonus plus a scaled bonus for users who meet the threshold.

4 Combined Results

Results from the pairwise and full-integration experiments will appear here once all three extension PRs (1a, 1b, 1c) have been merged and the integration PRs (4–7) are complete. The config matrix of seven experiment configurations is defined in `docs/initial_experiments/plans/state_action_reward_epic.md`. Currently available:

- `docs/initial_experiments/configs/1c_reward_only.yaml` (PR 1c, 2 actions, 2 states, multi-timescale reward) — see §3.